

Sample Chapter: Core JDO ISBN 0-13-140731-7

The chapters of the book titled “Core JDO” are being made available for the purposes of review to allow the community to participate in the review and editing of the chapters of the book.

Copyright (c) 2002 Sun Microsystems, Inc. All rights reserved.

FOR PRIVATE USE ONLY: This material is the property of Sun Microsystems, Inc. and is not to be sold, reproduced in any form by any means, or generally distributed without the prior written authorization of Sun Microsystems Inc and Prentice Hall.

To add your comments

- Simply send your uninhibited comments and feedback to the authors at jdoobook@yahoogroups.com
- Or click and add a note using Acrobat where needed. E-mail the PDF back to the authors at jdoobook@yahoogroups.com
- If you would like to receive the chapters in an alternate format, have any questions, concerns or please contact the authors at jdoobook@yahoogroups.com

“It is a capital mistake to theorize before one has data. Insensibly one begins to twist facts to suit theories, instead of theories to suit facts”-Sir Arthur Conan Doyle

JDO and JDBC

*T*his chapter will compare JDO with JDBC, another Java persistence API. JDBC is an earlier standard designed specifically to access relational database systems. While many readers may already be familiar with the JDBC API, for those that are not or would like to refresh their memories, the first part of this chapter outlines JDBC fundamentals, the second part will then compare JDO with JDBC.

JDBC 2.0 AND 3.0

Background

JDBC (“Java Database Connectivity”¹) is a Java API for accessing relational database systems. Prominent examples of relational database management system (RDBMS) products include Oracle, IBM DB2, Microsoft SQLServer, Sybase, Interbase, as well as open-source RDBMS solutions like MySQL or PostgreSQL.

Other approaches for database systems included ISAM, hierarchical, or document-oriented databases, as well as X.500/LDAP repositories, and, more recently, object databases. JDBC was not designed to access any of these non-relational database systems.

A relational database stores data entities in rows of tables, with fields of the entities stored as columns. Column values are constrained by an SQL data type, such as VAR-

¹ Sun's official position is that JDBC does **not** stand for Java Database Connectivity, possibly because of trademark issues with Microsoft's ODBC, Open Database Connectivity; however it is the generally accepted assumption and useful memory bridge to remember the acronym.

CHAR, INTEGER, TIMESTAMP, etc. Most tables will have one (or a combination of several) columns designated as primary key, which uniquely identify the row within (at least) the table. Relationships between entities are usually expressed by storing values of primary keys in columns of other entities, sometimes using foreign key integrity constraints.

Relational databases and thus usage through JDBC are closely modeled around a classical and simple client/server scenario: A client, e.g. an application using JDBC, sends a SQL request, e.g. a SELECT query statement, to a relational database server, which processes it and returns the result, e.g. a set of rows.

The query is expressed in SQL, the standardized Structured or Standard Query Language, of which SQL-92 is commonly implemented and the SQL-99 standard being the latest release. The SQL language includes keywords for querying, inserting and updating data, among others.

While the JDBC API as such is certainly object-oriented, using interfaces and classes, objects and methods, it is important to understand that it still is a (very thin, actually!) wrapper around, essentially, SQL, which is very procedural in nature. Persistent data in JDBC always comes in the row and column flavor, never directly as Java objects.

Other APIs

While JDBC today is the most widely used standard API to access relational databases using Java, alternative proposals exist, but have lost importance. The most well known example of such an alternative API is probably SQLj² (also known under the name of Oracle's implementation SQL4J, and formerly called JSQL), a joint venture between IBM, Oracle and other vendors. One of its ideas was to embed SQL code straight into Java source code, not inside Strings, and use a pre-compiler that checks that SQL and Java types match at application compile-time.

Using JDBC fundamentals

The basic steps using JDBC are always similar, and evolve around the following points:

1. obtain a `java.sql.Connection`, either from a `java.sql.DriverManager` or a `javax.sql.DataSource`. Using a `DataSource` is the preferred method today, and ensures portability to Application Servers using Connection Pools.
2. obtain a `java.sql.Statement` from the `Connection`, or re-use a `PreparedStatement` or `CallableStatement` possibly created earlier.
3. run SQL code, e.g. via the `executeUpdate("...")` method of `Statement`, or via the `executeQuery("...")` method, which returns a `java.sql.ResultSet`. (Note, the SQL can contain both Data Definition Language commands (DDL) like `CREATE TABLE` etc. as well as, more commonly, Data Manipulation Language commands (DML) like `INSERT`, `UPDATE`, `DELETE`, and `SELECT`)

² <http://www.sqlj.org>

4. if required, read data from the `ResultSet`, possibly iterating over it with the `next()` method if more than one row is returned in the set by the database server. Alternatively, do something else with the `ResultSet`, such as a `moveToInsertRow()`, `updateXYZ()`, `insertRow()`. This step may also be skipped if no data (of interest) was returned from the `Statement`
5. close the `ResultSet`, if one was used
6. close the statement
7. close the connection

Production code will have to do error handling via correct `SQLException` treatment (throws clause, catch and abort, catch and rethrow, etc.), as well as SQL warnings via the `getWarnings()` method of `Connection`, `Statements`, `ResultSet` etc.

The following UML sequence diagram illustrates the relationship between the major JDBC classes and interfaces:

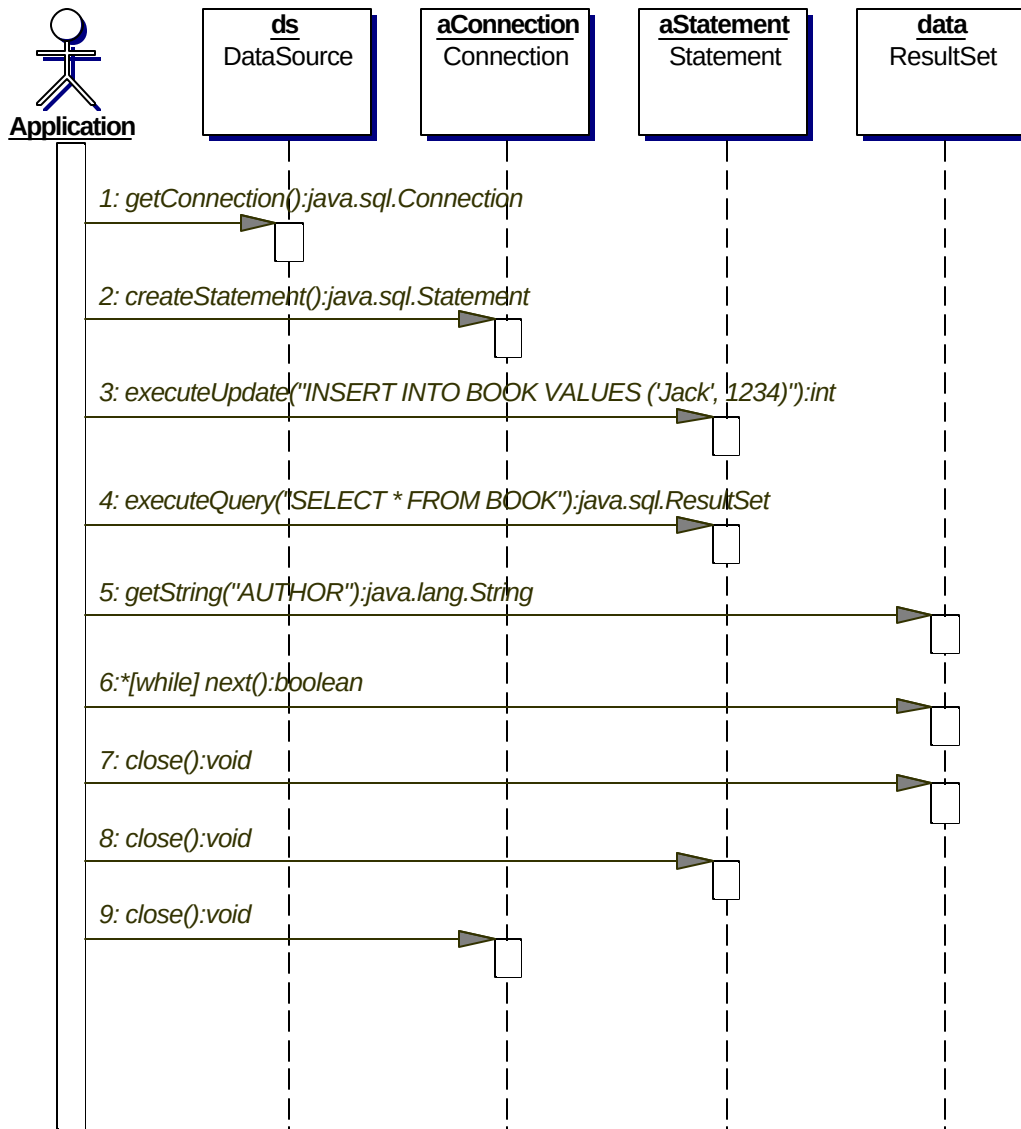


Figure 0-1 UML sequence diagram illustrating the major JDBC interfaces

JDBC History

The `java.sql` package, which provides the core of the JDBC API, was introduced as part of the core Java 1.1 API around January 1997.³

In May 1998, Sun released JDBC 2.0 with Version 1.2 of the Java 2 API and added a number of new classes to the `java.sql` package, including scrollable and updateable result sets, batch updates, BLOB and CLOB support, support for storing objects in their serialized form in relational database fields⁴, and introduced the `javax.sql` standard extension.

The `javax.sql` package includes additional features such as the `RowSet` class for treating database query results as JavaBeans, the `DataSource` interface used in connection pooling and to obtain Connections via JNDI. The package also provides support to allow JDBC clients to interact with the Java Transaction API (JTA) and perform units of work within existing (possibly distributed) transactions.

What's new in the JDBC 3.0 specification?

The latest JDBC specification at the time of writing is version 3.0, which was finalized in October 2001, and is also included in the “Merlin” Version 1.4 (JDK 1.4.x) of the Java 2 API. It does not aim at adding any revolutionary new concepts (such as maybe O/R mapping, non-relational data sources, XML, or anything like that) to the JDBC API. It does, however, provide numerous detail enhancements and new capabilities that introduce more flexibility and control for developers. These include, in order of perceived importance:

- New Savepoint interface, which contains new methods to set, release, or roll back a transaction to designated savepoints; e.g. `Savepoint svpt1 = conn.setSavepoint("SAVEPOINT_1"); (... ) conn.rollback(svpt1);`
- New configurable properties for the `ConnectionPoolDataSource` interface that can be used to describe how `PooledConnection` objects created by `DataSource` objects should be pooled. Furthermore, the specification documents how prepared statements are pooled and reused by connection pools.
- Added a means of retrieving values from columns containing automatically generated values; by appending the `Statement.RETURN_GENERATED_KEYS` flag to the `execute()`, `executeUpdate()` or `prepareStatement()` methods, the method `Statement.getGeneratedKeys()` can be called and will retrieve the value of such a key, as a `ResultSet` object with a column for each automatically generated key.

³ The original JDBC API (JDBC 1.0) was initially available as add-on to Java 1.0.

⁴ The `ResultSet.getObject()` and `PreparedStatement.setObject()` method do NOT perform any object/relational (O/R) mapping in any known JDBC implementation, but merely serialize and store a binary byte stream in a BLOB or `JAVA_OBJECT` type columns on which no query or referencing is possible!

- Added methods to alter the data contained in BLOB and CLOB objects. Also added of the `updateBlob`, `updateClob`, `updateArray`, and `updateRef` methods to the `ResultSet` interface.
- Added the data type `java.sql.Types.DATALINK`, allowing JDBC drivers to store and retrieve references to externally managed data, for example, data in a file outside the data source.
- Support for mapping SQL-99 user-defined (UDTs), structured and distinct types to classes in the Java programming language. Updated & new `DatabaseMetaData` methods for retrieving SQL type hierarchies.

Other minor enhancements include:

- New interface `ParameterMetaData`, which describes the number, type and properties of parameters to prepared statements.
- Added the new method `getMoreResults(int)` to `ResultSet`, which takes an argument that specifies whether `ResultSet` objects returned by a `Statement` object should be closed before returning any subsequent `ResultSet` objects.
- Added methods to allow a string to identify the parameter to be set for a `CallableStatement` object.
- Added the ability to specify the holdability of a `ResultSet` (cursor in DB) object.
- Added the data type `java.sql.Types.BOOLEAN` (logically equivalent to BIT).
- Added methods to retrieve the object referenced by a `Ref` object. Also added the ability to update a referenced object through the `Ref` object.

Last but not least, the JDBC 3.0 specification now includes a chapter clarifying the Relationship between the JDBC SPI (Service Provider Interface) and the Java Connector architecture JCA.⁵ JDBC driver vendors can now package their drivers as resource adapters and get all the benefits of pluggability, packaging, and deployment in Connector RAR File Format, and provide wrappers to implement the JCA system contracts. For more on JCA, see the separate chapter in this book on JCA and JDO.

Vendor-specific JDBC API extensions

While today JDBC is pretty much the universally accepted API to access relational databases from Java, see above, some vendors require API extensions that go beyond the JDBC standard. The generally accepted way of doing this is by casting an object returned from a standard JDBC method to a vendor-specific class, like in this example:

⁵ The Java Connector JCA architecture defines a standard way to package and deploy a “resource adapter” that allows a J2EE container to integrate its connection, transaction, and security management with those of an external resource.

```

ResultSet resultSet;
(...)
java.sql.Blob blob = resultSet.getBlob("BINARY");
if(blob instanceof oracle.sql.BLOB)
    return ((oracle.sql.BLOB)blob).getBinaryOutputStream();

```

EXAMPLE: STORING OBJECTS IN A RELATIONAL DATABASE USING JDBC

To put theory into practice, a complete JDBC mini-application example will be shown here. It uses the Book domain model used elsewhere in this book, and creates, modifies and displays books and authors. (Please note that this one class example is not a master piece of object oriented programming, but written to demonstrate basic JDBC usage with easy source code readability in mind.)

Running complete code will go here... tested and correctly indented etc. and broken up by short explanatory paragraphs. We'll "translate" a JDO example we use elsewhere, including the SQL schema (create_hsqldb.sql) that corresponds tables for the to the Author, Book, RevisedAuthor etc. classes, see diagrams below. Will use hsqldb and ant script to load fresh DB, thus providing a full example running out of the box.

Setup and obtain connection ...

```

package com.corejdo.chapter12_jdbc;
class SetupDatasource {

    static void main(String[] args) throws Exception {

        VendorDataSource vds = new VendorDataSource(); // ???
        vds.setServerName("my_database_server");
        vds.setDatabaseName("my_database");
        vds.setDescription("Core JDO book, JDBC chapter, sample
data source");

        Context ctx = new InitialContext();
        ctx.bind(datasourceName, vds);
    }
}

```



```
package com.corejdo.chapter12_jdbc;
class JDBCExampleDatasource extends JDBCExample {
    private String datasourceName;
    private DataSource ds;

    public JDBCExampleDatasource(String _datasourceName) {
        this.datasourceName = _datasourceName;

        Context ctx = new InitialContext();
        this.ds = (DataSource)ctx.lookup(this.datasourceName);
    }

    /**
     * Obtain a connection.
     */
    Connection getConnection() {
        // This has to be done every time:
        return ds.getConnection();
        // (jdbcConnectionUID, jdbcConnectionPwd); (?)
    }
}

package com.corejdo.chapter12_jdbc;
/**
 * CD/ZIP only, do not print!
 */
class JDBCExampleDriverManager extends JDBCExample {
    // @todo Do we have this sort of stuff elsewhere????
    private Properties Dbproperties = ...;
    private static final String DRIVER_CLASS_PROPERTY_NAME =
    "jdbc.driver.class";
    private String jdbcDriverClassName =
    Dbproperties.getProperty(DRIVER_CLASS_PROPERTY_NAME);
    ...

    public JDBCExampleDriverManager(...) {
        Class.forName(jdbcDriverClassName);
    }

    public Connection getConnection() {
        return DriverManager.getConnection( ... );
    }
}
```

```
package com.corejdo.chapter12_jdbc;
class JDBCExample {
    Setup and obtain connection, either from a DriverManager or a DataSource:

    /**
     * Obtain a connection.
     */
    abstract Connection getConnection();

    /**
     * not suited for a distributed or concurrent scenario
     */
    int getNewBookPK() {
    }

    /**
     *
     */
    void createBooks() {

        try {
            Connection con = this.getConnection();

            // Just for the fun of it, for the sake of example,
            // let's create the three books all
            // within the same Transaction, so:
            con.setAutoCommit(false);

            // GENERATE PK, WOULND'T BE NECC. WITH JDO.
            // Using other table, or MAX+1 each time?

            PreparedStatement createBookStmt = con.prepareStatement(
                "INSERT INTO BOOK VALUES (?, ?)");

            createBookStmt.setInt(1, 50);
            createBookStmt.setString(2, "Colombian");
            createBookStmt.executeUpdate();
            createBookStmt.close();

            // Create another book, and another one (3x)
            con.commit();
        }
        catch ( SQLException ex ) {
            if ( con != null ) {
```

```
        con.rollback();
    }
}
finally {
    if ( con != null ) {
        con.setAutoCommit(true);
        con.close();
    }
}
}

/**
 *
 */
void createRevisedBook() {
    // Show how complicated using a subclass is in JDBC;
    // make entry in both tables. (Should explain mapping
    // strategy, different possibilities, here or elsewhere?)

    // Transaction, have to created in one because of ref.int?
}

/**
 * Change the published date of given book.
 *
 */
void changeBookPublishedDate(String isbn, Date published)
    String updateSQL = "UPDATE BOOK SET PUBLISHED = ? WHERE
ISBN = ?";

...
}
```

Here is how to retrieve and print a list of all books:

```
/**
 * Prints all Books and their Authors to the console.
 */
void displayBooksWithResultSet() {
    Connection con = this.getConnection();
    // Do *NOT* con.setAutoCommit(false);

    // Not using a Prepared Statement, for once:
    Statement stmt = con.createStatement();
```

```

        ResultSet rs = stmt.executeQuery("SELECT * FROM BOOK");

        // Could show how to use a javax.sql RowSet here!!!

        while (rs.next()) {
            Date x = rs.getDate("a");
            String s = rs.getString("b");
            System.out.println(...);
        }

        con.close();
    }

    void displayBooksForRowSet() {
        RowSet rset = ...
        rset.setDataSourceName("jdbc/SomeDataSourceName");
        rset.setCommand("SELECT AUTHOR, ISBN FROM BOOK ");
        (...)
    }
}

```

Last but not least, here is how the above methods fit together:

```

/**
 * Standard Java main method, to run this class from CLI.
 */
static void main(String[] args) throws Exception {

    JDBCExample example = new JDBCExample(args);

    Dbproperties = ...

    example.createBooks();
    example.displayBooks();
    example.changeBook()
    example.displayBooks();
} // method
} // class

```

COMPARISON OF JDBC AND JDO

After having seen a JDBC example in this chapter, and JDO throughout the book, you will probably agree that the two APIs definitely “feel” very different to use. So what really makes this difference?

Well, JDO is a high-level API for transparent object persistence. The programmer can write code in the Java programming language and work with Java objects as always. The data of the objects is almost automatically made persistent and can be reused in a different run of the application, can be queried, and possibly accessed by other application; and all of this is achieved in a relatively transparent manner.

You may have noticed that the JDO API does not make any explicit reference on what the specification calls the “data store”, the place where objects are finally stored, queried and retrieved from. In some cases, a developer will choose to use a relational data store. In this case, the JDO implementation will generate the required SQL statements and translate JDOQL queries. This specific combination is often referred to as “object/relational mapping”: The developer uses objects, and some framework transparently maps access to objects into the relational world.⁶ Object/relational (O/R) mapping tools for Java have existed long before JDO, each with their own proprietary persistence API. In other cases, or at a later stage of a project, our same developer may choose to use a different data store, e.g. an object database, or an in-memory only store, or maybe a pure XML database.⁷ As JDO completely hides the SQL coding from the programmer, SQL generation is happening behind the scenes, there usually (see below) is no SQL in the business application itself, and swapping to a different data store is easily possible.

JDBC, on the other hand, is a lower-level API designed specifically to access relational databases, and does at no point attempt to “hide” the relational database in any way. In JDBC, developers explicitly code things like “update this information stored in this column of that table now”. Note that this is done using another language, SQL; statements of this language are embedded in Java Strings spread throughout the code, and sent to a RDBMS via JDBC. One has to realize that SQL, in all honesty, really has completely nothing to do with the language that was chosen to program the business application in, Java. This “disjoint” is often referred to as “impedance mismatch”.

The two APIs were very clearly designed with different goals in mind, and while both serve to “persist data” on a high-level, in practice they actually use fairly different means of achieving this goal. In a way there is limited scope for direct comparison.

The following sequence of diagrams further illustrates the difference. This UML class diagram shows some domain classes of a typical small application:

⁶ Names of tables and columns in the relational database are usually set in an XML mapping descriptor. This allows to easily change SQL table and column names, independently of the Java class and attribute name. The current JDO specification (1.0) does not standardize the format of this mapping description, presumably in the interest of staying 100% data store neutral, and each vendor uses similar but slightly different syntax. This may change with future releases of the JDO specification.

⁷ At the time of writing, the author was not aware of any commercially available JDO implementation using a pure real XML database as data store; however this is likely purely a question of time.

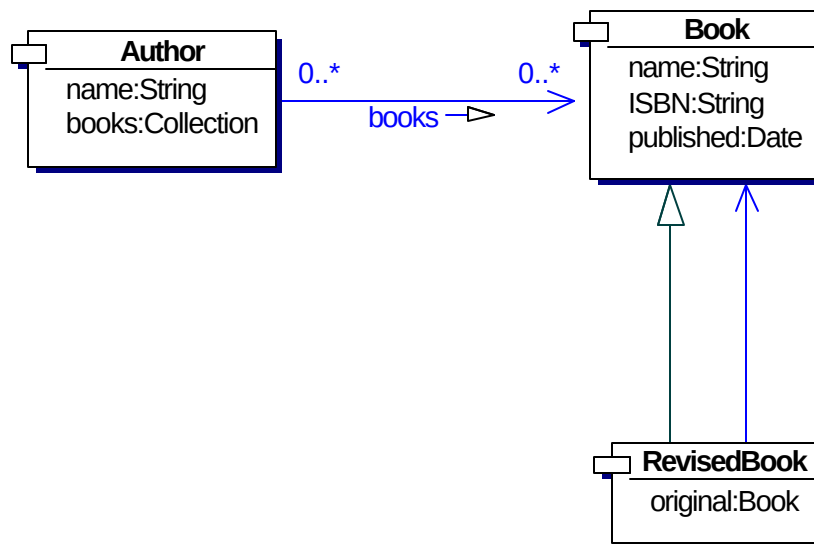


Figure 0-2 UML class diagram

It represents the Java object-oriented view of the business domain: A Book has an ISBN number, here modeled as an attribute of type String, so the Book class will generally have a `setISBN()` and `getISBN()` method. Some books are special; they are subclasses of general books, so there is a RevisedBook Java subclass that inherits from Book. Authors publish books, so the Author class has a books attribute with e.g. a `getBooks()` method which returns some `java.util.Collection` of Book objects. A new author is created using the new `Author()` constructor, just like any other object in Java is instantiated:

```

Author author1 = new Author();
author1.setName("Donald Duck");
pm.makePersistent(author1);

```

Using JDO, working with authors and books in a persistent database will mean working directly with these actual domain objects, and with the JDO API for calls like `makePersistent()` and executing queries expressed via a Query language (JDOQL) that is syntactically very close to Java code syntax, e.g. using navigational expressions and familiar methods like `Collection.contains()`, `Collection.isEmpty()`, `String.startsWith()` etc. For example:

```

String filter = "original.name == \"Donald Duck\"";
Query query = pm.newQuery(RevisedBook.class, filter);
Collection result = (Collection) query.execute();

```

```
revisedBook = (RevisedBook) result.iterator().next();
query.close(result);
```

With JDO, it really doesn't (have to) go much beyond this. Many JDBC-only based business applications today will also have domain data object classes similar to the above in the Java application. However, to work with authors and books in a persistent database in this case will mean working on a different view, that of relational databases. The following logical ER diagram illustrates how authors and books live in an RDBMS:

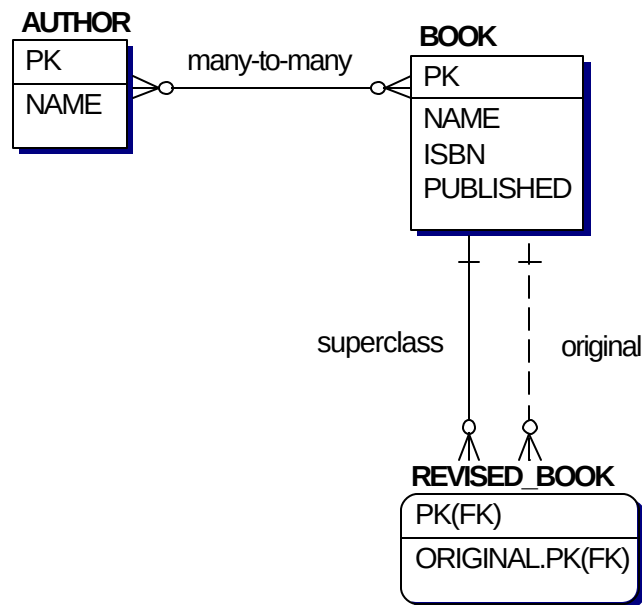


Figure 0-3 Logical ER (Entity-Relationship) diagram

While this logical diagram may still look fairly similar to the above UML class diagram, JDBC code will work with SQL at the level of a physical ER diagram, where e.g. what's expressed as a "many-to-many" relationship in a relational model becomes the intermediate **AUTHOR_BOOK** table, and where Strings are VARCHARs:

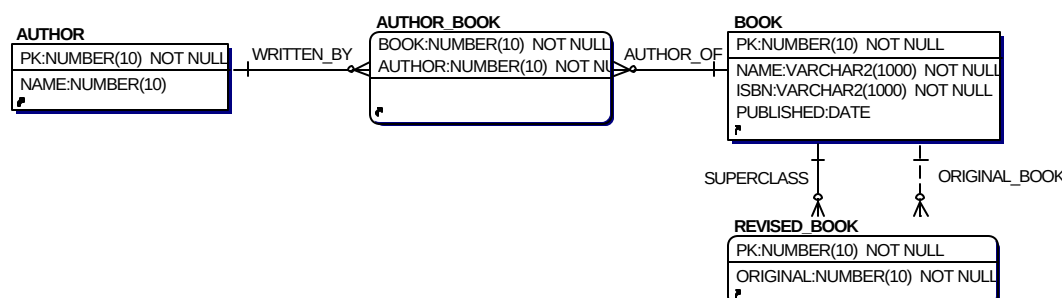


Figure 0-4 Physical ER (Entity-Relationship) diagram (e.g. for Oracle)

In this view, there are no object relationships, nor any queries “following” an object reference; there are tables, rows to be INSERTed, columns to be SELECTed, object references that became NUMBER types, and SQL queries to be formed with JOIN and other SQL keywords. In which model would you rather work?

In brief review, JDO scores much higher on programmer convenience. JDO hides SQL from the programmer; that is, a developer using the Java programming language does not need to learn SQL. On the other hand, the JDBC API provides greater control by giving programmers direct access to the relation database access.

Feature comparison of JDBC vs. JDO

After the above discussion, following a brief one-on-one feature comparison of JDBC and JDO, in so far as the two APIs can actually be compared; as mentioned, they were designed for different purposes.

Table 0-1 Feature comparison of JDBC and JDO

Feature	JDBC (alone)	JDO
Transparent object persistency	No	Yes
Object references support	No	Yes
Persistence by reachability	No	Yes
Class inheritance support	No	Yes
Transactional (JTS/JTA)	Yes	Yes *
Lazy/deferred/partial loading	No, manual	Yes
Locking	Manual	Automatic *
Standard Java API	Yes, java.sql (core package)	Yes, javax.jdo (extension package)

Feature	JDBC (alone)	JDO
Platforms applicable	J2SE & J2EE ‡	J2SE, J2EE & J2ME ready
Formally part of J2EE	Yes	No, but... not an issue †
Maturity	Fairly mature	Recent, but rapidly adopted
Developer community	Large	Small, but growing
Compile-time type checking	Limited	Better
Connection pooling possible	Yes	Yes
Data Caching	None	Usually yes, various *
Scalability	Limited	Cluster with distributed cache *
Data stores	Relational only	Independent API, today often relational or native object, basically completely open, including EAI
Query Language	SQL only	JDOQL, possibly others (SQL*)
DDL / meta data access	SQL / yes	No, data store dependant
RDBMS Views, Stored Proc.	Yes, sure	No (but *)
Renaming tables and columns	Often hard-coded	Easily changed in descriptors *
Persistence-related code	Often scattered and intermixed with application business logic §	Separate by design

* JDO implementation vendor dependant

† JDO fits into the larger J2EE picture as a JCA resource adapter

‡ JDBC can not be used on the Java Micro Edition (J2ME) platform.

However, a similar but much simplified non-standard API is available on J2ME devices.

§ Unless elaborate design patterns such as DAO are implemented. While of course it is technically possible to cleanly separate persistence-related code from domain model and business logic classes, the point is that with direct JDBC programming, this requires good architecture and discipline. With JDO, the separation comes for free, out of the box.

Regarding the point on developer community: It is only natural that JDO as a new emerging Java technology has a smaller experienced user base than JDBC that has been around since almost the beginning of Java, and has been widely used. However, as mentioned, JDO merely standardizes an API for transparent object persistency, for which products have existing for some time. Senior enterprise developers often have knowledge from O/R products and will be able to adopt JDO easily. Junior developers usually find JDO easy to adopt because it's intuitive and by design extremely close to "normal" Java. Sometimes people realize that they have in fact been writing an in-house persistent framework and are only too keen on jump on a now standardized API.

In summary, for many Java applications, technologies that solve the impedance mismatch between an Object Model and a Data Model are of strong interest, and far too often redeveloped in one form or the other in many projects. Transparent Persistency, Persistence By Reachability, and Object to Relational (O/R) Mapping, Lazy/deferred/partial loading and caching are highly desirable features for many applications. JDO-compliant implementations provide these features out of the box, and are thus an interesting alternative to direct JDBC programming.

Misconceptions about JDBC and JDO

JDO is not a replacement for JDBC

“Using JDO, I can completely forget about JDBC!” – Uh, careful. While it is true that with JDO it is now perfectly possible to develop simple applications entirely without reference to the JDBC API in the application code, you may want to think twice before throwing away your JDBC and SQL documentation.

Firstly, as long as you are in a O/R mapping scenario where you want to use a relational database as the final data store for your JDO-based application, you or at least some people on a project’s development team will still be dealing with Connection URLs or Data Sources, JDBC drivers, Connection Pools, and the like. In this scenario, JDO does not really replace JDBC, but isolates business application developers from its details, allowing to code at a higher level, while a JDO implementation internally will still use JDBC and a JDBC driver to access the relational data store at the lower level. (Think of this as a bit like using JSPs at a higher level that simplifies Servlet programming, even though JSPs are still translated into Servlets; similarly, in JDO many operations will ultimately lead to JDBC calls.) Of course, if you use a non-relational data store, this will be different.

Secondly, for more sophisticated enterprise applications, sometimes a mix of JDO with special cases relying on some JDBC is required, and you may well hang on to those PreparedStatements and ResultSets for specific modules of your applications for some time to come. See the “When to use JDBC” chapter below for more details.

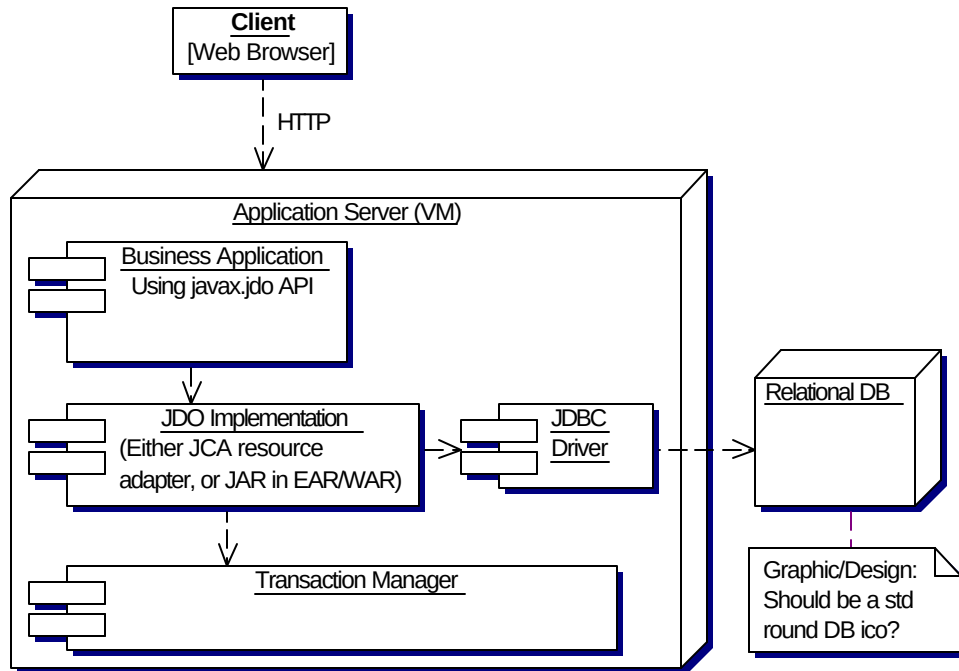


Figure 0-5 Layers of a typical web-based application when using JDO on a relational JDBC-based data store within an Application Server

It is best to think about JDO as a complementary technology to JDBC, both with unique strengths, usable by programmers with different skill sets and applicable to projects or modules of projects with different development objectives.

Performance

“JDBC is faster than JDO!” – No, not necessarily. To compare the performance of a JDO-based application with a JDBC-based application, a lot depends on the actual application scenarios, specific use case and details of how JDBC is used. Here are some of the factors that contribute to the performance discussion:

- **Mapping:** When reading and writing persistent objects from and to a data store, a JDO implementation is usually required to do some sort of mapping between the Java instances and the objects in the data store. For example, when using a relational JDBC-based data store, at some point in time, a JDO implementation will have `ResultSet` and want to set the attributes of a data object from it. While this mapping will always consume time, what’s more important is that during

the mapping, data that the application may not need may unnecessarily be read or written and mapped. Depending on the sophistication of an alternative JDBC-based application, this point can actually come out in favor of JDO, instead of against it, see the item on unnecessary reading below. Furthermore, the example as explained above only applies to relational data stores; other JDO implementations that are e.g. using an object database as data store may have a simpler (or no!) mapping function and this point does not apply. In the overall performance picture, mapping does not generally lead to a significant performance penalty, taking into account that in many JDBC-based applications there also is some sort of mapping done, sometimes more hidden (i.e. spread all over the code) or more explicit (if using home written) in application code.

- **Unnecessary reading:** When reading data from a data store, performance obviously depends on how much data is read, and how. A JDBC-based application may use a specific SQL statement for each specific read access, or some more generic (SELECT *, JOIN in case we may need the data later) construct. JDO implementations, on the other hand, are able to only fetch data from a data store that is actually required, thus minimizing database hits and network traffic. Today, this is usually implemented using a technique referred to as Deferred Reading, wherein an JDO implementation will only fetch some data initially (see the default-fetch-group attribute of the field tag in JDO XML metadata descriptors) and fill in other attributes only later, only if actually accessed. More sophisticated mechanisms can be envisioned, e.g. a Prefetch API that is being discussed for future versions of the JDO standard, which would allow the application to specify a policy whereby instances of persistence capable classes will be pre-fetched from the datastore when related instances are fetched; or automatic detection of reading patterns to optimize fetching in advanced JDO implementations. The bottom of this point is “it depends”, performance could be in favor of JDO versus a simple generic JDBC application, or against JDO versus a sophisticated JDBC application with different handcrafted specific queries. However, JDO could gain a big point in simplicity, for the same performance, because a JDO application developer does not have to be concerned with datastore fetch details, while with JDBC, the developer writes the SQL code and has to think ahead about unnecessary reads.
- **Unnecessary writing:** When writing data to a data store, most JDO implementations today are able to only write modified fields, because they automatically keep track of write access to fields of persistent objects. Furthermore, if optimistic transaction management is used, many JDO implementations are able to “batch” writes, to avoid writing fields that are changed again within the same transaction, to avoid any data store round trip if a transaction is aborted, and other optimizations based on leveraging the in-memory object model. These points can again contribute to minimizing database cycles and network traffic.
- **Caching:** By far the most important aspect of performance discussions between JDO and straight JDBC is caching, as any experienced developer will confirm.

Most JDO implementations keep a global cache of objects, according to different, vendor specific and often configurable, caching strategies. Therefore, in many JDO applications, it will not be required to read and map an object previously fetched again, because it is kept in a memory cache in the JVM by the JDO implementation. Note that for a JDO application developer, this caching is fully transparent; there is no “load from cache instead from datastore” etc. code in the application. Again, the impact of this depends on the actual application, amount of data, available memory and use case; in general however this leads to a huge performance benefit in favor of the usual JDBC application that sends of a new query statement each time data is required. (Note that using prepared statements and similar “optimizations” are of minimal impact in the bigger picture of having an actual domain object cache in memory.) There is little to no performance penalty for using a cache, however, there is an impact on memory requirements.

Of course, all of the above points regarding performance could be implemented in one form or the other in a JDBC-based application as well; they are not technically bound to JDO as such. However, your general JDBC applications will often not implement caching, lazy load/write etc. strategies. If they do, you have in fact developed an in-house persistence layer that’s similar to what’s now available from various vendors, according to a standard API. See the “build versus buy” discussion earlier in this book for more about this.

Database independence

“JDBC abstracted differences between relational databases, and now JDO abstracts even access to non-relational data stores!” – Let’s have closer look at the details:

- While the SQL 92 standard exactly defines the syntax of the SQL language, and a simple “`SELECT * FROM table`” certainly works across all relational databases, vendors often implement more advanced requirements differently. Many projects that need to work across different relational databases thus end up accounting for differences in the underlying database’s SQL in the Java code of the business application, or in some in-house written database abstraction layer.
- While JDBC driver classes abstract (necessarily, because there is no standard) the product-specific client/server connection protocol details (e.g. Oracle OCI, MySQL port 3306 protocol, etc.) JDBC does not make any attempt to “translate” SQL code.⁸ JDO implementations however translate the datastore inde-

⁸ The JDBC 3.0 Specification suggests that “Because scalar functions are supported by different DBMSs with slightly different syntax, it is the driver’s job either to map them into the appropriate syntax or to implement the functions directly in the driver.”; in practice, however, most JDBC drivers even from established vendors do not seem to respect such requirements as of today.

pendent JDO query language (JDOQL) according to the underlying datastore, and could take slight SQL differences into account and map accordingly.

- The O/R-mapping based JDO implementations often provide this database product abstraction layer; and it is in fact not unusual to find some sort of vendor-specific "database type" configuration property. Most JDO vendors will also list which relational databases their implementation has been explicitly tested and approved against. So to switch from one relational database to another, for example from a simple Java embedded database running in the same VM to a full-fledged remote relational DB from one of the leading vendors, the same JDO implementation, configured differently, can often be used without too much trouble. While such claims will naturally sound suspicious to anybody who has tried this with JDBC and become disappointed, this does generally work fairly well with JDO in practice.

For these reasons, using different relational databases from one project is easier with JDO which hides possibly database vendor specific SQL in favor of a data store neutral query language than with JDBC. If non-relational data stores, such as object databases, are a possible future target, JDBC is not an option as persistence API anyway.

Switching JDO implementation to switch data store?

Note that sometimes a switch from one type of datastore to another one, e.g. from relational datastore to a very scalable object database data store, sometimes or even from one to a different vendor of the same type of datastore, e.g. from Oracle to DB2, will require a switch in the JDO implementation you have chosen.

As the market stands today, JDO implementations are relatively clearly split into "for relational database" versus the "for (often same vendor's!) object databases" products. Of course, switching JDO vendor will only be possible without rewriting code if only the standard JDO API has been used; using JDO vendor extensions will "lock" you into that particular vendor. As long as only the standard API has been used, switching vendors is generally "relatively" easy, because the JDO specification is very specific and strict, and implementations can pass a technology compatibility kit (JDO TCK) from Sun.

When to use JDBC after all

Some applications may need to implement use cases where JDBC is required after all. Often these are not your trivial example application, but some medium to complex solutions. Broadly, this situation often falls into one of these cases:

- ? Specific SQL queries that can not be expressed in JDOQL
- ? Need to use existing often vendor specific relational datastore features
- ? Data-centric, processing intensive, performance-sensitive (batch) applications

? Existing relational database schemas, potentially complex and not modifiable

SQL queries

It is important to understand that the JDO standard Query Language, JDOQL, does *not* attempt to be a drop-in replacement for SQL. Its aim is not to be capable of expressing any SQL query one can possibly think of.

The most common and widely used SQL statements such as INSERT, UPDATE, SELECT, DELETE of course have equivalents in the JDO API, as well as many SELECT options such as WHERE and ORDER BY have equivalents in JDOQL. Similarly, the BEGIN, SET TRANSACTION, COMMIT, ROLLBACK and related statements have equivalents as JDO Transactions.⁹

Other SQL constructs however have no direct equivalent in the JDO API or JDOQL. This is "a feature, not a bug", in the sense that JDOQL was conceptually not meant to suit such needs. And while minor extensions may be made in the future, most of such constructs will probably never be expressible using JDOQL. These include:

- SELECT options like GROUP BY and HAVING (and aggregate functions such as AVG, COUNT DISTINCT, MAX, MIN, SUM) have no direct equivalent in JDOQL.
- SELECT options JOIN, IN and EXISTS as well as sub queries do not directly translate into JDOQL. However, it needs to be understood that some of these SQL operations, particularly IN and JOIN, may not be required to express the same construct in JDOQL, simply because it's closer to the object model. A good example for this is JOIN operations in typical relational database applications on helper tables used to express multi-multi (M:N) relationships: In a JDO model, this is expressed as Collection attributes of the respective data object, and queried using .contains() operations, or simple navigation through a reference field, which gets translated to the correct JOIN operations in SQL, if running on a relational datastore. At the JDOQL level, explicit mention of the intermediate tables and JOINS is simply not required any more.
- Unions and intersections (SELECT ... UNION/INTERSECT/EXCEPT SELECT ... constructs) cannot be expressed directly in JDOQL. These SQL keywords are not formally part of the SQL standard. Similarly to the above comment, the same goal can often be achieved differently using JDO. Specifically, sometimes such constructions are used to "hack" in SQL what would be correctly modeled as class inheritance in a JDO application.

⁹ These transaction-related statements are never issued as such even in JDBC; the JDBC driver usually issues these (or uses another DB-proprietary transaction demarcation mechanism) after each individual SQL statement as soon as it is complete, if auto-commit is enabled, or when the client code calls Connection.commit() or Connection.rollback() if auto-commit is disabled on the the Connection.

- Some operators, most importantly string functions like the SQL LOWER, UPPER, SUBSTRING, TRIM, LIKE, as well as case-insensitive string comparison, are not standardized by the JDOQL specification, which mandates only support for the `String.startsWith()` and `String.endsWith()` methods. However, JDO implementations frequently provide vendor extensions to use LIKE and/or “%” syntax. Make sure you understand and highlight in your code when you are using such vendor extensions. Future releases of the JDO specification may define more supported String methods for JDOQL query filters.
- SQL commands like CALL for stored procedures, or TRIGGER related functions, are not covered by JDO. These statements are often very database specific.
- Last but not least, schema manipulation statements such as CREATE/ALTER/DROP of TABLE or INDEX are not directly supported by the JDO specification. These Data Definition Language (DDL) statements are not usually executed at the run-time of an application, but during development and build. JDO implementations usually provide tools that create and/or upgrade the required database schema, which do issue these statements.

So when looking at an existing application with SQL code, and evaluating JDO migration, it is important to work on a query-by-query basis, before prematurely concluding something like “this application has queries that are too complex and can not be expressed in JDOQL.”

If after all your application does requires some SQL, this is not necessarily a reason for the entire application to be written using JDBC instead of JDO. In practice, there are three basic choices to use SQL queries in JDO applications:

1. Use JDBC API to execute SQL query in JDO application
2. Use JDO standard to send off a pure SQL query via the JDO API. As seen in the Query chapter, the JDO specification deliberately lets a developer specify the query language when creating a JDO Query object, and while it's usually JDOQL, several implementations available today offer the possibility to “pass through” full SQL code. (Note that the expected result set still generally has to be a collection of persistent objects.)
3. Use JDO vendor specific extension to mix JDOQL and SQL syntax in one and the same query filter. This can be used with some implementations to e.g. “pass through” relation datastore specific SQL query hints and the like, if really needed.

Specific relational datastore features

Regarding the use of specific relational datastore features, often vendor proprietary SQL extensions or entire applications, consider this example: A document/content management

solution wants to use JDO. It will have domain classes to represent Document, Author, Version etc. However, it also needs to interface to a database specific full text indexing engine, for which vendor specific SQL calls are required.

This in itself is not a reason to develop the entire application in JDBC; it is perfectly legal to use JDBC/SQL for that part of your application. As explained above, the choices are either to use SQL as alternative query language from the JDO API, if the JDO implementation chosen permits this, or simply through good old real JDBC API calls. See below for an example of how to correctly use both APIs mixed to perform transactional work.

Data-centric applications vs. domain-model centric

Other than these technical issues, it is sometimes also argued that while JDO is great for domain-model centric applications, other applications lend themselves more for JDBC than JDO. For example, heavily data-centric processing, including decision support, ad-hoc query and reporting applications that may be interested to offload processing using existing relational database features.

Existing legacy relational database schemas

Some applications are required to work on an existing legacy database schema. If such database schemas are actually complex relational database “applications”, e.g. using SQL views (instead direct SELECT on underlying tables) and stored procedures (instead direct INSERT) to access data, raw JDBC may be a better fit than JDO. Although some vendors do claim to actually e.g. Stored Procedures, it is generally not possible to “map” an arbitrary complex relational schema to an object world.

However, simpler and straight-forward existing database schemas may be understood and correctly mapped by a good JDO implementation. For example:

- **Identity Mapping:** Just because the standard schema generator of your JDO implementation of choice generates a let’s say auto-incrementing sequence number as primary key, this does not automatically mean it could not also use e.g. a FirstName / LastName pair that models the primary key in an existing schema. This could be modeled with a multi-attribute Application Identity class in JDO.
- **Simple Field Mapping:** While, yes, String attributes usually are mapped to VARCHAR columns, int etc. to NUMBER columns, etc., most JDO implementations can be flexible about this. For example, mapping a VARCHAR(1) column with ‘Y’ / ‘N’ in a table to a Boolean is usually possible. Often more complex mappings (using application-provided translators) can be implemented.
- **Collection Field Mapping:** In relational modeling, either a one-to-many or a many-to-many mapping expresses what in a Java class model is usually both written as a Collection-type attribute: With a one-to-many mapping, the objects contained in the collection are linked to exactly one (or none) containing object,

but never more than one; this is modeled with a foreign key on the contained records associating to the containing record and corresponds to what UML refers to as an aggregate link. With a many-to-many mapping, the objects added to the Collection-type attribute can be linked by many of the same objects; this is modeled with an intermediate table holding the foreign keys of both records. Because a pure Java class model does not (need to) distinguish these two cases, most O/R-based JDO implementations will by default use a many-to-many mapping for Collection-type attribute. However, most JDO implementations are perfectly able to work with one-to-many mapping as well. (Note, the relational one-to-one mapping is less ambiguous and simply corresponds to a persistent object reference in the respective class)

- **Inheritance Mapping:** Another area is relational schemas which use the same primary key values in more than one table, aiming at expressing some sort of “this record actually is an ‘extension’ of that record” relationship. While this could be mapped into two separate classes with an association, such relational structures often map well into a class model using inheritance.
- **JDO specific tables and columns:** O/R-mapping implementations often require extra tables or columns. For example, a column on each table to keep an incrementing locking value used to implement optimistic transaction management, and one holding the name of the class of a persistence object used to properly interpret subclasses. If such columns are not part of an existing schema, some implementations permit to disabled their usage if the required feature (e.g. optimistic transactions, or subclassing) is of no interest.

Some O/R-based JDO implementations provide schema reverse engineering tools, promising to generate Java classes and JDO metadata with mapping information from an existing database schema. This could greatly simplify (or at least support) such a mapping process as outlined above, as opposed to manual configuration.

Last but not least, consider that it may be quite acceptable to enhance an existing schema for a new application, without however changing its structure. For example the point on JDO specific columns would only really apply if the legacy schema is completely non-modifiable, which may not always be the case; i.e. slight additions to a database may be acceptable. Sometimes even more profound schema changes of a legacy database may be perfectly acceptable, e.g. because no legacy application will continue to use it in parallel with a new, JDO-based application, then the schema can be completely adapted and data migrated, and thus easily used with JDO anyway. The above few paragraphs should have illustrated however that even if a legacy schema can not be changed, it may be worth to investigate if a JDO implementation can actually work with that schema.

Last but not least, it should be expected that this area will see some interesting developments in the future, with JDO implementations becoming more mature, and able to tackle some of the issues.

Data access by other applications

Sometimes it is not the application to be developed that is the element causing doubt between a JDO vs. JDBC choice, but existing external applications that should interface with the application to be written. Reporting tools are a typical candidate: most industry-standard reporting tools as of today access data at the relational level, via JDBC or other relational APIs like ODBC or native drivers, and are only slowly beginning to gear up to be able to leverage object domain model representations of business data. However, this should generally not prevent a new application from being written in JDO, because if a relational data store is chosen, other applications will still be perfectly able to access the data:

This approach generally works relatively well, specifically if only read access is required, as is usually the case for reporting tools. In case data structures from JDO need to be simplified for human report designers, e.g. when using several tables for inheritance mapping, then creating some SQL views for the non-JDO application may simplify things, and works well.

However, if the other application requires write access to the same data, the scenario can get more complex because of cache synchronization. This issue, however, is not really strictly related to JDO, and would equally arise with a JDBC-based application that attempted to cache data. Possible solutions include disabled caches, which is perfectly possible with many JDO implementations, or some sort of manual cache refreshing or synchronization.

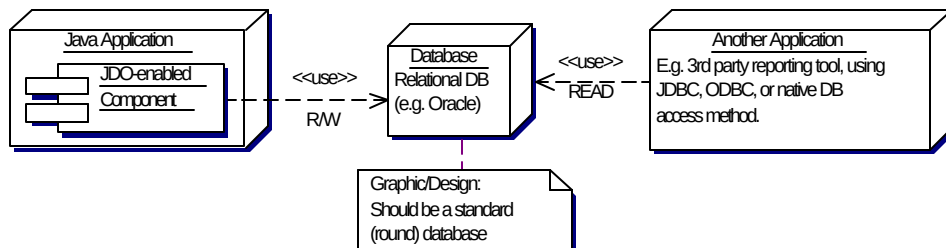


Figure 0-6 JDO application and other application both accessing relational datastore

When to use JDO and JDBC in an application

So it may be possible for your application to completely use JDO, instead of JDBC. You may after all wish to use only JDBC, instead of JDO. It really all depends on the application in question. And sometimes you may wish to use mixed JDO code, with some JDBC still.

While such a mixed architecture should be avoided unless there is a good reason for it, and among other issues severely limits portability to another datastore, it is technically certainly possible. One approach is to use vendor-specific API extensions to e.g. use SQL instead of (or mixed with) JDOQL as query language, as described above. Another road,

depending on why the application desires to use a mixed architecture, is to use a vendor-specific method to obtain some JDBC-class instance to retrieve some information or perform some operation; e.g. obtaining a `JDBC Connection` instance from a `JDO PersistenceManager`.

A different road, and as opposed to the above a portable and clean (albeit slightly more complicated) architecture, is to perform some work within the same transaction, on a different or maybe even the same data source. Both JDO and JDBC calls can be made within a `JTA UserTransaction` in a managed environment. The Transaction chapter of this book explains this topic and its background, shows how to obtain a PM inside a JTA transaction, and presents a complete example illustrating how to use JDO alongside another transactional resource such as JDBC.

SUMMARY

This chapter has reviewed JDBC, another API for persisting data in Java, specifically tailored to wide-spread relational databases. A brief overview of relational databases, history of JDBC, introduction to the new JDBC 3.0 API, source code case study showing the practical use of JDBC were presented in the beginning of the chapter.

The second part of the chapter then compared JDBC with JDO, discussed features and differences of each, highlighted common misconceptions. The general conclusion drawn was that they are complementary APIs, each serving their own purpose well, and examples of applicability of each API were discussed.

On performance, it was shown that a JDO-based application need not be slower than a JDBC-based one, and in many cases can actually be faster because unnecessary but very costly remote round-trips to the datastore could be avoided.

On JDBC 3.0 innovations, the most interesting point from a JDO perspective may be the new support for mapping SQL-99 user-defined types (UDTs) to classes in the Java programming language; future will tell whether relational database vendor will consistently implement this in their products and JDBC drivers, and JDO implementations are actually able to leverage this new JDBC 3.0 feature and transparently map persistent JDO-enhanced classes to SQL-99 UDTs, instead of having to implement the `SQLData` interface and writing to `SQLOutput` and `SQLInput` streams manually, as would be required with a pure-JDBC 3.0 application.

Last but not least, we think that after you have read this book, the cases in which you will want to use only JDBC, with no JDO, for a new usual general-purpose application, will be rare ones. Special "processing" like application for bulk loading, data transformation etc. come to mind as may not suitable for JDO at all.